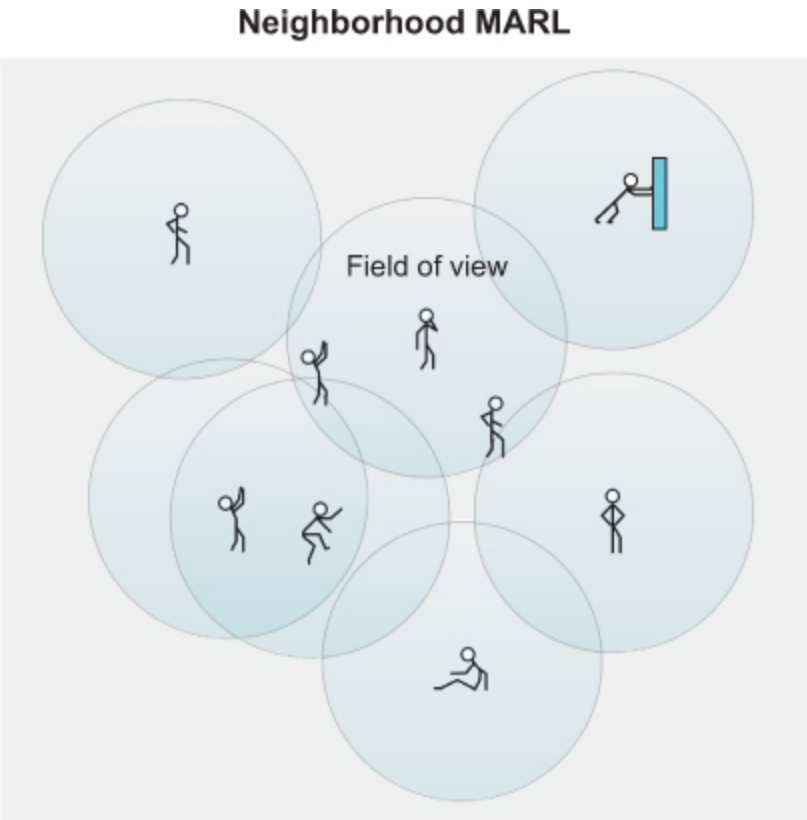


By constraining the size of the neighborhood, we stop the exponential growth of the joint-action space to the fixed size we set for the neighborhood. If we have a multi-agent Gridworld with 4 actions for each agent and 100 agents total, the full joint-action space is 4^100, which is an intractable size; no computer could possibly compute with (or even store) such a large vector. However, if we use subspaces of the joint-action space and set the size of each subspace (neighborhood) to 3 (so the size of each subspace is $4^3 = 64$), this is a much bigger vector than with a single agent, but it's definitely something we can compute with. In this case, if we're computing the Q values for agent 1, we find the 3 agents closest in distance to agent 1 and build a joint-action one-hot vector of length 64 for these 3 agents. That's what we give to the Q function (figure 9.7). So for each of the 100 agents, we would build these subspace joint-action vectors, and use them to compute Q values for each agent. Then we would use those Q values to take actions as usual.

Figure 9.7. The neighborhood Q function for agent j accepts the current state and the joint-action vector for the other agents within its neighborhood (or field of view), denoted a_{-j}. It produces Q values that get passed to the policy function that chooses the action to take.